MongoDB Technical Report 1

1. Introduction

My chosen distributed platform/framework for the project is MongoDB because I have previously worked with it on a previous project. I also feel Mongo is easier to use and understand than other frameworks.

2. Environment Setup

For the setup of the distributed system, I am using Docker to host 3 containers each running MongoDB. These containers are using a replica set to synchronize their databases.

- Network configuration: All containers are connected to a single virtual network
- RAM Maximum Usage per container: 7.37 GB
- CPU usage: 1.30%

3. Installation and Functional Verification

For the setup, first we install Docker on our machine, which runs EndeavourOS, an Arch-based Linux distribution. In order to install Docker, we first run the command “sudo pacman –S docker” which downloads and install all the needed dependencies.

Once everything has been installed, we run Docker to make sure it has installed correctly. If no errors occur, then we need to write a Docker compose file which sets up 3 container images running MongoDB and a mongo-express container to help visualize the database better on a browser dashboard.

Below is a screenshot of the docker compose file:

/ root / Documents / ≡ docker-compose.yaml

```yaml
# docker-compose.yml
version: "3.8"

services:
  mongo1:
    image: mongo:latest
    container_name: mongo1
    hostname: mongo1
    restart: unless-stopped
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports:
      - "27017:27017"
    volumes:
      - mongo1_data:/data/db
    networks:
      - mongo-cluster

  mongo2:
    image: mongo:latest
    container_name: mongo2
    hostname: mongo2
    restart: unless-stopped
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports:
      - "27018:27017"
    volumes:
      - mongo2_data:/data/db
    networks:
      - mongo-cluster

  mongo3:
    image: mongo:latest
    container_name: mongo3
    hostname: mongo3
    restart: unless-stopped
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports:
      - "27019:27017"
    volumes:
      - mongo3_data:/data/db
    networks:
      - mongo-cluster

  mongo-express:
    image: mongo-express:latest
    container_name: mongo-express
    restart: unless-stopped
    ports:
      - "8081:8081"
    environment:
      ME_CONFIG_MONGODB_URL: mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0
    depends_on:
      - mongo1
      - mongo2
      - mongo3
    networks:
      - mongo-cluster

volumes:
  mongo1_data:
  mongo2_data:
  mongo3_data:

networks:
  mongo-cluster:
    driver: bridge
```

Once the docker compose file is written, we open a terminal and navigate to the folder where the file resides. When we enter the folder, we can execute the command “docker compose up -d”. This will prompt docker to download the mongodb and mongo-express images and then set up the containers with their names, ports, volumes and network.

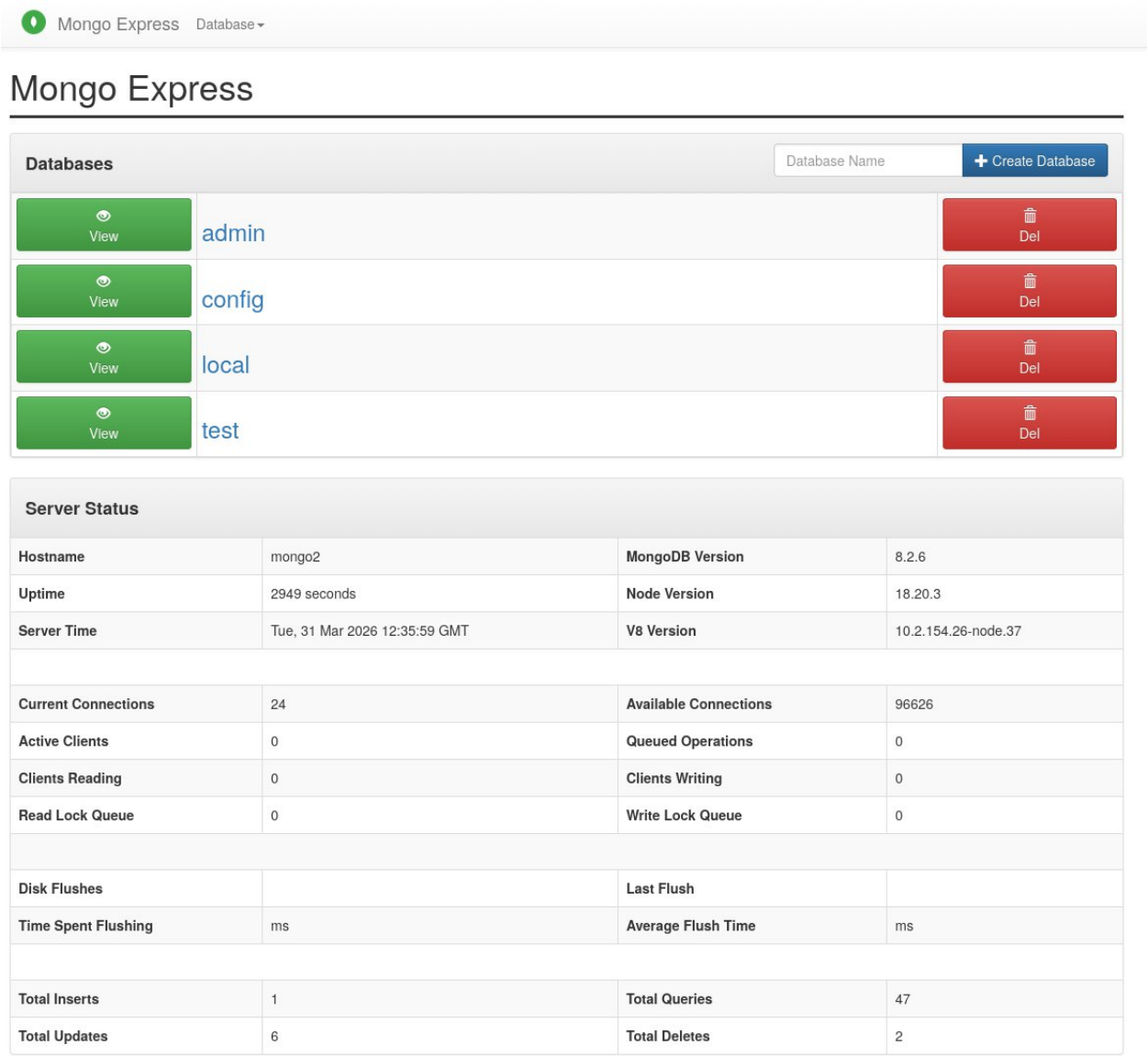
This part of the setup has been quite challenging as we had to write the docker compose file ourselves, something which I, personally, haven’t done before. It took a day or two to read through documentations of both Docker and MongoDB to set up this composer file and to make sure everything worked smoothly.

After Docker finishes setting up the containers, we can run the command “docker exec –it mongo1 mongosh” which lets up connect to the first MongoDB container in the mongo shell. Once here, we run the following command:

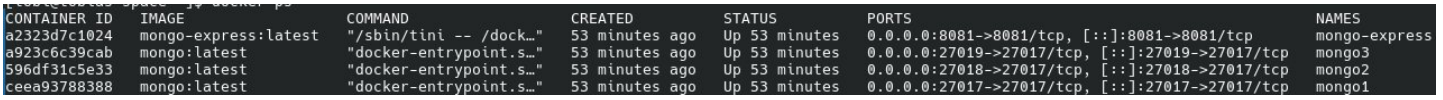
```
rs.initiate({
        _id: "rs0",
        members: [
        { _id: 0, host: "mongo1:27017" },
        { _id: 1, host: "mongo2:27017" },
        { _id: 2, host: "mongo3:27017" }
         ]
})
```

This will set up our replica set which allows our 3 MongoDB containers to use and sync a single database from multiple instances of MongoDB

To visualize the database, we can use Mongo Express to connect to a dashboard on our browser at the address localhost:8081.



For testing that everything works properly, we use “docker ps” to check if every container is up and running without any problems



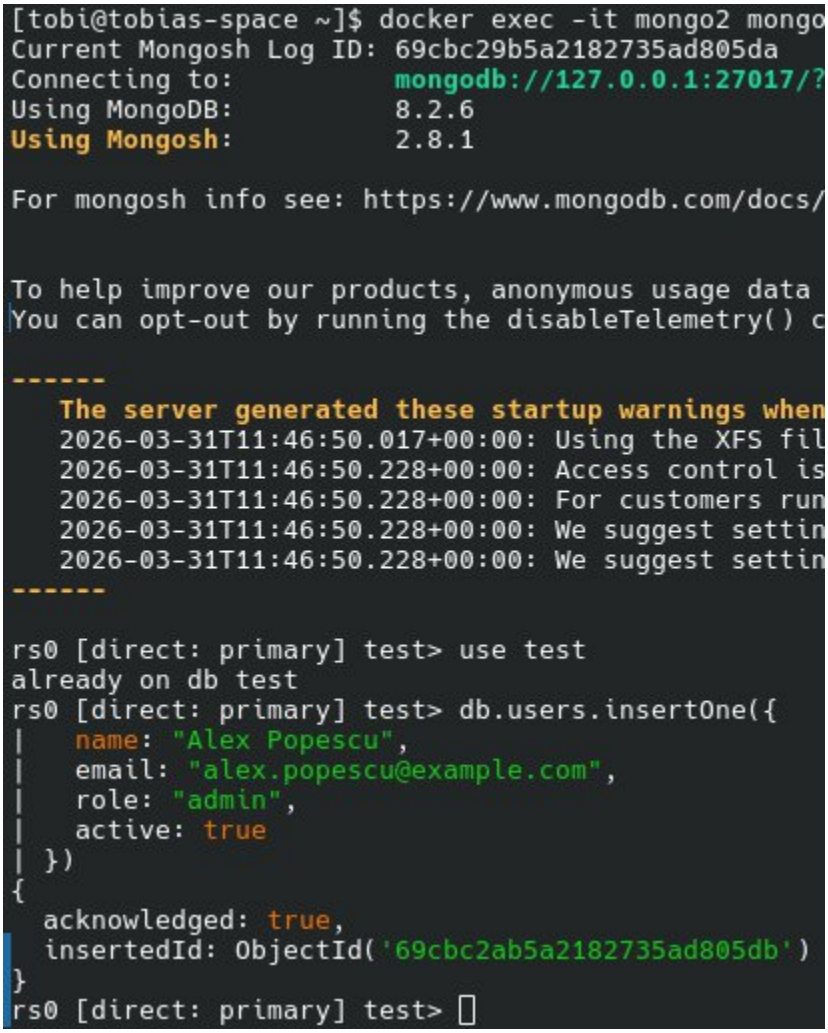
In the screenshot, we can see that each container is working properly and has been up for some time. To make sure that the replica set is also set up properly, we use the command rs.status() to tell us the details:

```
rs0 [direct: secondary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-03-31T12:43:45.001Z'),
  myState: 2,
  term: Long('2'),
  syncSourceHost: 'mongo2:27017',
  syncSourceId: 1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    lastCommittedWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    appliedOpTime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    durableOpTime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    writtenOpTime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    lastAppliedWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastDurableWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastWrittenWallTime: ISODate('2026-03-31T12:43:40.741Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1774960970, i: 1 }),
  electionParticipantMetrics: {
    votedForCandidate: true,
    electionTerm: Long('2'),
    lastVoteDate: ISODate('2026-03-31T11:47:00.293Z'),
    electionCandidateMemberId: 1,
    voteReason: '',
    lastWrittenOpTimeAtElection: { ts: Timestamp({ t: 1774957578, i: 1 }), t: Long('1') },
    maxWrittenOpTimeInSet: { ts: Timestamp({ t: 1774957578, i: 1 }), t: Long('1') },
    lastAppliedOpTimeAtElection: { ts: Timestamp({ t: 1774957578, i: 1 }), t: Long('1') },
    maxAppliedOpTimeInSet: { ts: Timestamp({ t: 1774957578, i: 1 }), t: Long('1') },
    priorityAtElection: 1,
    newTermStartDate: ISODate('2026-03-31T11:47:00.298Z'),
    newTermAppliedDate: ISODate('2026-03-31T11:47:00.304Z')
  },
  members: [
    {
      _id: 0,
      name: 'mongo1:27017',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 3415,
      optime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
      optimeDate: ISODate('2026-03-31T12:43:40.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
      optimeWrittenDate: ISODate('2026-03-31T12:43:40.000Z'),
      lastAppliedWallTime: ISODate('2026-03-31T12:43:40.741Z'),
      lastDurableWallTime: ISODate('2026-03-31T12:43:40.741Z'),
      lastWrittenWallTime: ISODate('2026-03-31T12:43:40.741Z'),
      syncSourceHost: 'mongo2:27017',
      syncSourceId: 1,
      infoMessage: '',
      configVersion: 1,
      configTerm: 2,
      self: true,
      lastHeartbeatMessage: ''
    },
```

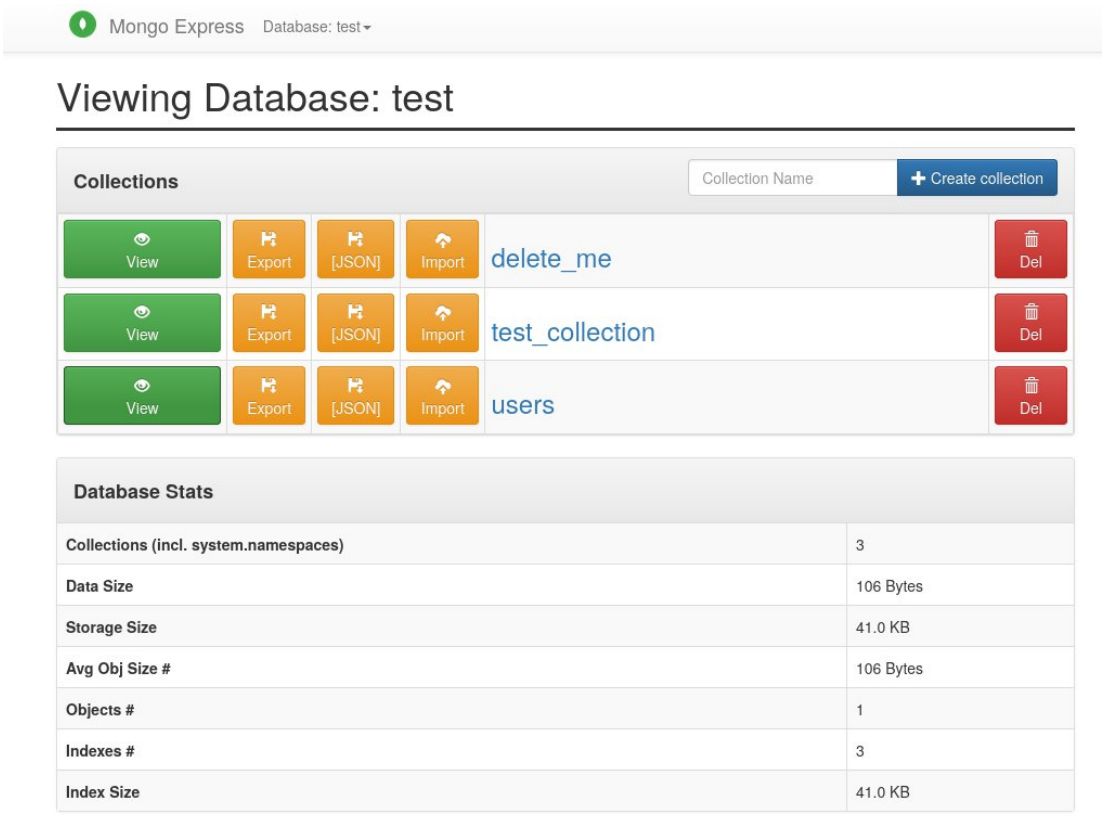
```
  {
    _id: 1,
    name: 'mongo2:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 3414,
    optime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-03-31T12:43:40.000Z'),
    optimeDurableDate: ISODate('2026-03-31T12:43:40.000Z'),
    optimeWrittenDate: ISODate('2026-03-31T12:43:40.000Z'),
    lastAppliedWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastDurableWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastWrittenWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastHeartbeat: ISODate('2026-03-31T12:43:43.422Z'),
    lastHeartbeatRecv: ISODate('2026-03-31T12:43:44.887Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1774957620, i: 1 }),
    electionDate: ISODate('2026-03-31T11:47:00.000Z'),
    configVersion: 1,
    configTerm: 2
  },
  {
    _id: 2,
    name: 'mongo3:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 3414,
    optime: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1774961020, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-03-31T12:43:40.000Z'),
    optimeDurableDate: ISODate('2026-03-31T12:43:40.000Z'),
    optimeWrittenDate: ISODate('2026-03-31T12:43:40.000Z'),
    lastAppliedWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastDurableWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastWrittenWallTime: ISODate('2026-03-31T12:43:40.741Z'),
    lastHeartbeat: ISODate('2026-03-31T12:43:44.886Z'),
    lastHeartbeatRecv: ISODate('2026-03-31T12:43:43.425Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'mongo2:27017',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 2
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1774961020, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1774961020, i: 1 })

s0 [direct: secondary] test>
```

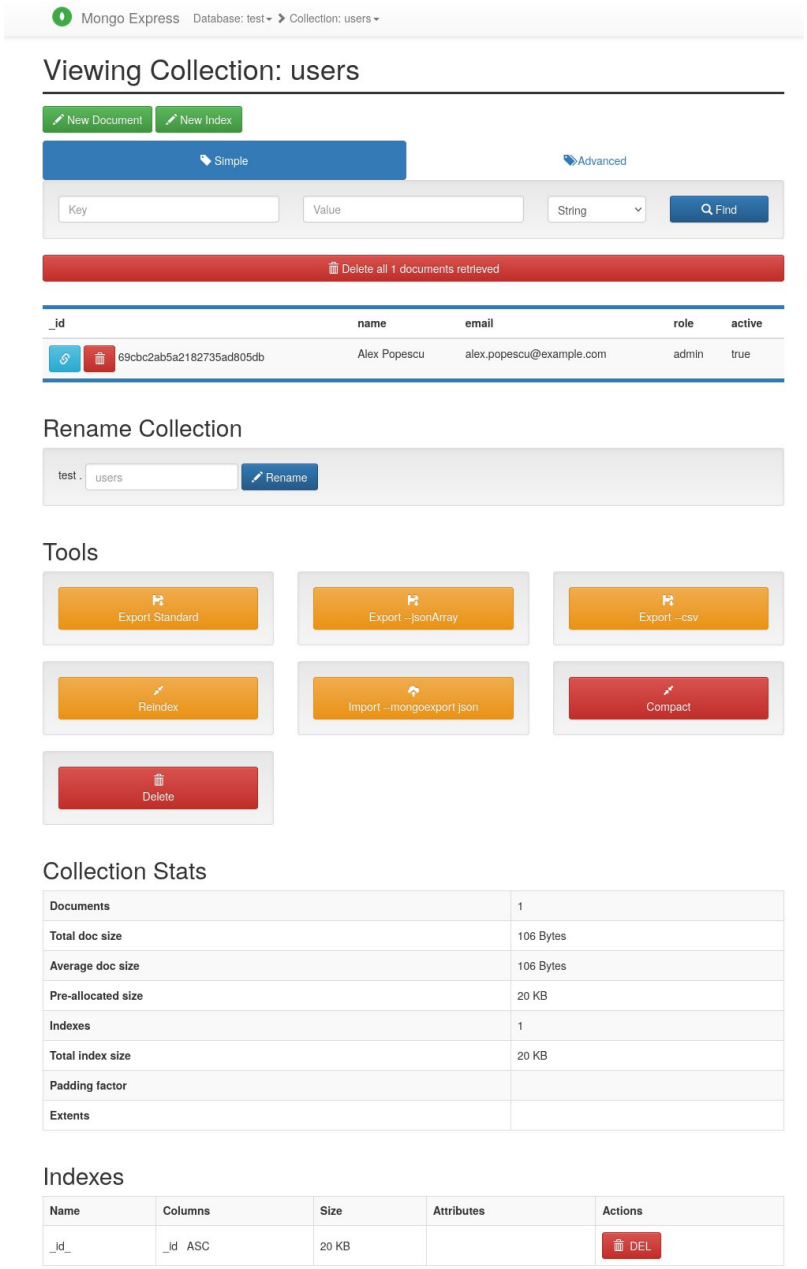
In these screenshots, we can see the details of the replica set and the details for each member.

When testing the addition of an entry to the database, we can see everything is working fine:

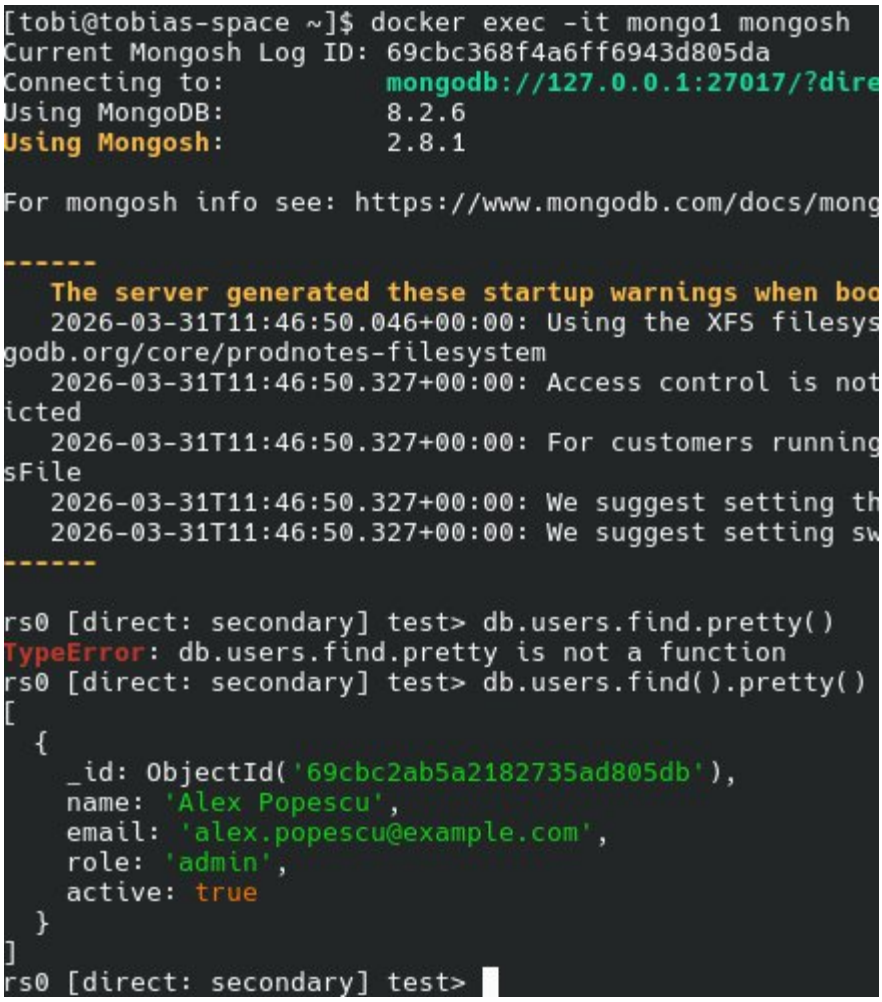


And the result:





As we can see from the browser interface, our new entry has been successfully added to the database. And if we try to read the entry with another user:



We can find it easily using the find() command. And in the case that the primary user falls (we stop the main container forcefully for testing):

```
rs0 [direct: secondary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-03-31T13:00:57.629Z'),
  myState: 1,
  term: Long('3'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
    lastCommittedWallTime: ISODate('2026-03-31T13:00:54.009Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
    appliedOpTime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
    durableOpTime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
    writtenOpTime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
    lastAppliedWallTime: ISODate('2026-03-31T13:00:54.009Z'),
    lastDurableWallTime: ISODate('2026-03-31T13:00:54.009Z'),
    lastWrittenWallTime: ISODate('2026-03-31T13:00:54.009Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1774962040, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'stepUpRequestSkipDryRun',
    lastElectionDate: ISODate('2026-03-31T13:00:53.999Z'),
    electionTerm: Long('3'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1774962050, i: 1 }), t: Long('2') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1774962050, i: 1 }), t: Long('2') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1774962050, i: 1 }), t: Long('2') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    priorPrimaryMemberId: 1,
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-03-31T13:00:54.009Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-03-31T13:00:54.019Z')
  },
  members: [
    {
      _id: 0,
      name: 'mongo1:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 69,
      optime: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
      optimeDate: ISODate('2026-03-31T13:00:54.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1774962054, i: 2 }), t: Long('3') },
      optimeWrittenDate: ISODate('2026-03-31T13:00:54.000Z'),
      lastAppliedWallTime: ISODate('2026-03-31T13:00:54.009Z'),
      lastDurableWallTime: ISODate('2026-03-31T13:00:54.009Z'),
      lastWrittenWallTime: ISODate('2026-03-31T13:00:54.009Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1774962054, i: 1 }),
      electionDate: ISODate('2026-03-31T13:00:54.000Z'),
      configVersion: 1,
      configTerm: 3,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: 'mongo2:27017',
      health: 0,
      state: 8,
      stateStr: '(not reachable/healthy)',
      uptime: 0,
      optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
      optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
      optimeWritten: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
```

When connected to mongo1, which is in a secondary state initially, we can see it is promoted to primary since mongo2 is down and he used to be the primary. Now, when we try writing as mongo1:

We can clearly see that it we are allowed to write without any problems. When we start mongo2 back up, it starts as secondary and mongo1 remains the primary that can write in the database:

```
members: [
  {
    _id: 0,
    name: 'mongo1:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 273,
    optime: { ts: Timestamp({ t: 1774962254, i: 1 }), t: Long('3') },
    optimeDate: ISODate('2026-03-31T13:04:14.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1774962254, i: 1 }), t: Long('3') },
    optimeWrittenDate: ISODate('2026-03-31T13:04:14.000Z'),
    lastAppliedWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    lastDurableWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    lastWrittenWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1774962054, i: 1 }),
    electionDate: ISODate('2026-03-31T13:00:54.000Z'),
    configVersion: 1,
    configTerm: 3,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 1,
    name: 'mongo2:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 2,
    optime: { ts: Timestamp({ t: 1774962254, i: 1 }), t: Long('3') },
    optimeDurable: { ts: Timestamp({ t: 1774962254, i: 1 }), t: Long('3') },
    optimeWritten: { ts: Timestamp({ t: 1774962254, i: 1 }), t: Long('3') },
    optimeDate: ISODate('2026-03-31T13:04:14.000Z'),
    optimeDurableDate: ISODate('2026-03-31T13:04:14.000Z'),
    optimeWrittenDate: ISODate('2026-03-31T13:04:14.000Z'),
    lastAppliedWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    lastDurableWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    lastWrittenWallTime: ISODate('2026-03-31T13:04:14.035Z'),
    lastHeartbeat: ISODate('2026-03-31T13:04:20.949Z'),
    lastHeartbeatRecv: ISODate('2026-03-31T13:04:20.613Z'),
    pingMs: Long('0'),
```

With all these tests done, we have assured ourselves that the system is running as intended and does not fail if one of the devices fails.

4. Conclusion:

In conclusion, the MongoDB distributed framework has been installed successfully and without any problems, all the tests passing without any problems. We proved that there are no errors while running this setup and we also showed how the system still works even if a node fails.